

CgToolkit

Einführung in die Nutzung



Grundgerüst Code:

Die Software nutzt C++ mit OpenGL (GLFW, GLEW) und kann entweder zur Anzeige von 3D Szenen oder für Testbilder (umgesetzt als Texturen) für die Bildverarbeitung verwendet werden. Bedienelemente werden durch ImGui realisiert, das ist eine Gui, die direkt im Verzeichnis mit compiliert wird und für die keine Installation notwendig ist. Der OpenGL Teil ist fertig implementiert, Kenntnisse in OpenGL sind zur Bearbeitung der Aufgaben nicht nötig. Änderungen und Ergänzungen am Code müssen nur in sehr wenigen Klassen gemacht werden.

Im File `main.cpp` wird einfach entschieden, ob 3D-Rendering oder Bildverarbeitung gemacht werden soll. Es ist entweder oder möglich.

```
CgGLFWwindow mainWindow;  
mainWindow.init(window_size_x,window_size_y);  
  
// setup Gui for OpenGL Rendering  
CgOpenGLRenderingGui myGui(current_path);  
mainWindow.setGui((CgBaseGui*)&myGui);  
  
// setup Gui for Image Processing  
// CgImageViewer myGui(current_path);  
// mainWindow.setGui((CgBaseGui*)&myGui);  
  
mainWindow.show();
```

Benötigte Libraries

Generell werden die Libraries OpenGL, glew, glfw3, glm und cmake benötigt. Unter einem neu installierten „blanken“ Ubuntu 24.04 hat folgendes funktioniert:

```
sudo apt install g++ libglew-dev libglm-dev libglfw3-dev cmake
```

Compilieren / cmake:

Das Programm kann mit Hilfe von cmake übersetzt werden. Dazu wechseln Sie in den Folder `build` und tippen `cmake ../src`. Das Erzeugt ein Makefile in `build`, welches Sie mit `make` nutzen können um zu kompilieren. Das Programm `CgToolkitMain` liegt dann ebenfalls im `build` Ordner.

Für MacOS waren zusätzliche include Pfade in `CMakeLists.txt` nötig. Hier müssen Sie ggf. Anpassungen vornehmen für ihr System.

Ausführen:

Tippen Sie `./CgToolkitMain` im Ordner `build`. Sie können die Fenster-Größe per Parameterübergabe festlegen (z.B. `./CgToolkitMain 600 300` legt die Größe auf 600x300 fest). Wenn das zu nervig ist, können Sie die entsprechenden Variablen in `main.cpp` für Ihre Zwecke anpassen. Die Größe der Bereiche für die Bildanzeige und die Bedienelemente passen sich automatisch an.

Erzeugen von Project-Files

Mit dem Kommando `cmake -GXcode ../src` kann ein Project-File für Xcode erzeugt werden. Dieses kann dann einfach in XCode geöffnet werden und sollte übersetzen.

Für Eclipse wird die Erzeugung mit dem Kommando `cmake -G "Eclipse CDT4 - Unix Makefiles" ../src` nicht mehr empfohlen (deprecated). Hier gibt es `cmake4eclipse` im Eclipse Marketplace. Allerdings habe ich das nicht getestet.

ImGui

ImGui ist eine so genannte Immediate Mode GUI, d.h. konkret dass alle Grafik-Primitive direkt vom Client in die Grafik-Pipeline eingehängt werden und ähnlich wie im RenderLoop von OpenGL immer wieder in jedem Frame gezeichnet und aktualisiert werden. Das bedeutet, dass relativ einfach neue Gui-Elemente hinzugefügt werden können und die Werte der Gui-Elemente (slider values etc.) recht einfach abgefragt werden können.

Die Grundfunktionalität eines GUI-Elements ist relativ gut in folgenden Methoden zu sehen:

- **Für OpenGL-Rendering:**

in `CgOpenGLRenderingGui::createRenderOptionsGui()`. Hier werden verschiedene Gui-Elemente erstellt und deren Werte ausgelesen.

- **Für Bildverarbeitung:**

in `CgImageViewer::createAufgabe1Tab()`. Hier werden auch einige Gui-Elemente erstellt und deren Werte ausgelesen.

Man kann z.B. den Wert einer Checkbox wie folgt setzen: angenommen wir haben, um die Beleuchtung ein- und auszuschalten, eine Variable `bool m_lighting_mode` mit einem entsprechenden Gui-Element verknüpft:

```
ImGui::Checkbox("enable lighting", &m_lighting_mode);  
m_renderer.setLightingMode(m_lighting_mode);
```

Dadurch dass die Checkbox im RenderLoop immer wieder aktualisiert wird, hat die Variable `bool m_lighting_mode` zu jeder Zeit den aktuell eingestellten Wert. Dieser wird dann mit der nächsten Zeile (in jedem Render-Loop neu) für den OpenGL-Renderer gesetzt. Der Render-Mode wird also immer wieder gesetzt und nicht nur wenn er geändert wird.

Wenn mit Betätigung eines Gui-Elements eine Aktion ausgeführt werden soll (und zwar nur dann), so wird dies durch den Rückgabewert der jeweiligen Funktion initiiert. Eine Abfrage und gleichzeitig Anzeige eines Buttons über

```
if (ImGui::Button("reset"))  
{  
    reset();  
}
```

zeigt also den Button an, und stellt gleichzeitig den Callback für die Ausführung der entsprechenden Aktion bei Betätigung des GUI-Elements dar, falls diese erfolgt ist. Beide Funktionalitäten gemeinsam, d.h. If-Abfrage für Callbacks und setzen von Werten ist auch möglich, etwa bei Slidern

```
if (ImGui::SliderInt(" ", &m_linewidth, 0, 30))  
{  
    my_image.drawCross(...,m_linewidth)  
}
```

Wichtig: Nahezu alle ImGui Widgets lassen sich auf diese Weise nutzen.

Für die Bearbeitung der Aufgaben wird es nötig und nützlich sein, dass Sie sich selbst eigene Bedienelemente erstellen. Machen Sie sich mit der Funktionsweise von ImGui in der Klasse `CgOpenGLRenderingGui` bzw. `CgImageViewer` vertraut und erstellen Sie dort für einige eigene ImGui Widgets und verknüpfen Sie diese mit entsprechenden Variablen, so dass Sie bei Änderung/Betätigung des Gui-Elements eine entsprechende Aktion starten können (z.B. testweise eine Methode, die eine Bestätigung auf `std::cout` schreibt). Die jeweiligen Variablen werden als Member der Klasse `CgOpenGLRenderingGui` bzw. `CgImageViewer` realisiert.

- Eine sehr ausführliche Übersicht von möglichen Gui-Elementen finden sie im Demo-Fenster rechts unten in der Anwendung.
- Den entsprechenden Code dazu finden Sie in `./imgui/imgui_demo.cpp`. Hier können Sie sich Überblick und Anregung für mögliche Gui-Elemente holen.
- Wenn Sie diese Übersicht nicht mehr benötigen können Sie sie per flag `show_demo_window` im Konstruktor von `CgOpenGLRenderingGui` bzw

`CgImageViewer` abschalten. Das Fenster mit den Tabs für die Aufgaben wird dann automatisch größer.

Für Bildverarbeitung:

Generell reicht es aus, wenn Sie nur Gui-Elemente in `CgImageViewer::createAufgabe2Tab()` und analog für die Tabs der anderen Aufgaben einfügen, die restlichen Struktur ist schon vorhanden.

Für OpenGL Rendering:

Es reicht, wenn Sie Elemente in `CgOpenGLRenderingGui::createAufgabenTabBar()` einfügen, die restliche Struktur ist schon vorhanden. Bei Bedarf können Sie Elemente zu `CgOpenGLRenderingGui::createRenderOptionsGui()` hinzufügen.